Homework 2

16824 Visual Learning and Recognition (Fall 2026)
https://piazza.com/cmu/fall2026/16824/

RELEASED: Wednesday, 7th Oct. 2026
DUE: 11:59 PM ET, Wednesday, 4th Nov. 2026
Instructor: Jun-Yan Zhu
TAs: Jenny Wang, Matthew Sivaprakasam, Avigyan Bhattacharya, Silong Yong, Shen Zheng

START HERE: Instructions

- Collaboration policy: All are encouraged to work together BUT you must do your own work (code and write up). If you work with someone, please include their name in your write-up and cite any code that has been discussed. If we find highly identical write-ups or code or lack of proper accreditation of collaborators, we will take action according to strict university policies. See the Academic Integrity Section detailed in the initial lecture for more information.
- Late Submission Policy: There are a total of 5 late days across all homework submissions. Submissions that use additional late days will incur a 10% penalty per late day.
- Submitting your work:
  - We will be using Gradescope (https://gradescope.com/) to submit the Problem Sets. Please use the provided template only. You do not need any additional packages and using them is strongly discouraged. Submissions must be written in LaTeX. All submissions not adhering to the template will not be graded and receive a zero.
  - Deliverables: Please submit all the .py files. Add all relevant plots and text answers in the boxes provided in this file. To include plots you can simply modify the already provided latex code. Submit the compiled .pdf report as well.

NOTE: Partial points will be given for implementing parts of the homework even if you don't get the mentioned accuracy as long as you include partial results in this pdf.

1 Generative Adversarial Networks (50 points)

We will be training Generative Adversarial Networks (GAN) on the CUB 2011 Dataset.

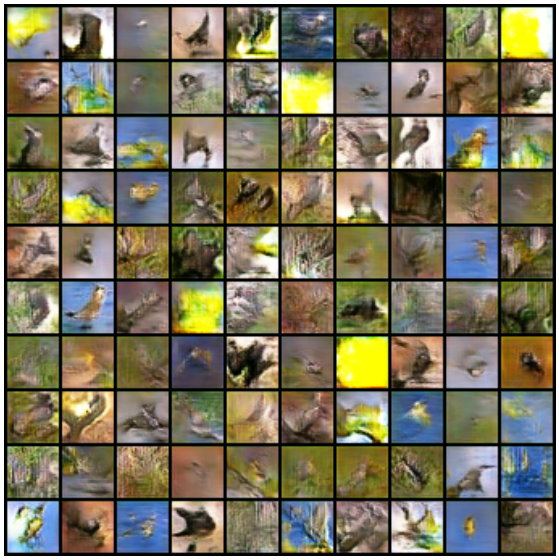
- Setup: Run the following command to set up everything you need for the assignment: ./setup.sh /path/to/python_env/lib/python3.8/site-packages.
- Question: Follow the instructions in the README.md file in the gan/ folder to complete the implementation of GANs.
- Debugging Tips:
  - GAN losses are pretty much meaningless! If you want to understand if your network is learning, visualize the samples. The FID score should generally be going down as well.
  - Do NOT change the hyper-parameters at all, they have been carefully tuned to ensure the networks will train stably. If things aren't working its a bug in your code.
  - For debugging, disable JIT using export PYTORCH_JIT=0 python ... and disable AMP by using the flag –disable_amp. However, do note that disabling JIT will cause the FID calculation to fail. So only disable JIT to make sure that your network code runs correctly, then re-enable when training. If you observe any errors involving type mismatches and tensors that have half types, it is due to AMP, you may need to explicitly cast the tensor using .half().
  - Here is a sample image from WGAN-GP at the end of training. The other networks may have variations but should look similar:



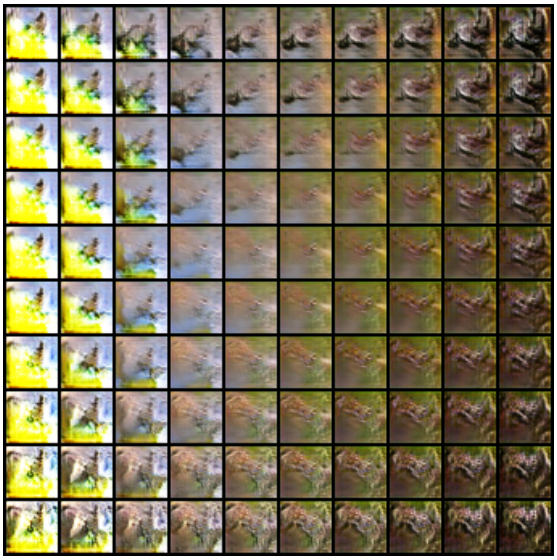
- Expected results:
  - Vanilla GAN: Final FID should be less than 110.
  - LS-GAN: Final FID should be less than 90.
  - WGAN-GP: Final FID should be less than 70.
- Deliverables: The code will log plots to gan/data_gan, gan/data_ls_gan, and gan/data_wgan_gp. Extract plots and paste them into the appropriate section below. Note for all questions, we ask for final FID. Final FID is computed using 50K samples, at the very end of training. See the final printout for "Final FID (Full 50K)".

1. Paste your plot of the samples and latent space interpolations from Vanilla GAN as well as the final FID score you obtained.

Solution:
Vanilla GAN FID: 73.91 (50,000 samples).



(a) Vanilla GAN Samples



(b) Vanilla GAN Latent Space Interpolations

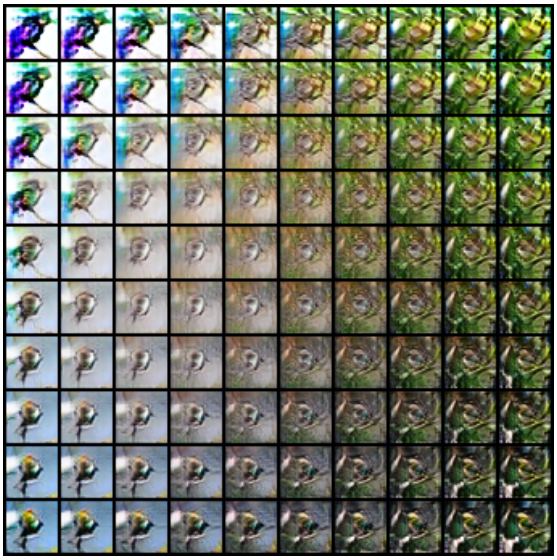
The samples capture varied background colors and several recognizable bird shapes, but many images are blurry and some have bright color artifacts. Nearby vectors in the interpolation grid produce gradual changes in color and texture, although bird shapes are not consistently clear.

2. Paste your plot of the samples and latent space interpolations from LS-GAN as well as the final FID score you obtained.

Solution:
LS-GAN FID: 53.74 (50,000 samples).



(a) LS-GAN Samples



(b) LS-GAN Latent Space Interpolations

Many samples contain recognizable bird shapes against varied backgrounds, but some still have mottled textures and distorted body parts. The interpolation grid shows gradual changes in appearance and background between nearby latent vectors.

3. Paste your plot of the samples and latent space interpolations from WGAN-GP as well as the final FID score you obtained.

Solution:
WGAN-GP FID: 33.50 (50,000 samples).



(a) WGAN-GP Samples (b) WGAN-GP Latent Space Interpolations

The sample grid contains clearer bird outlines and more coherent backgrounds than the Vanilla GAN grid, although some poses and fine details are still distorted. The interpolation grid changes gradually in background and bird appearance between nearby latent vectors.

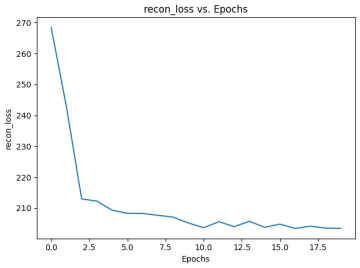
2 Variational Autoencoders (30 pts)

We will be training Autoencoders and Variational Auto-Encoders (VAE) on the CIFAR-10 dataset.

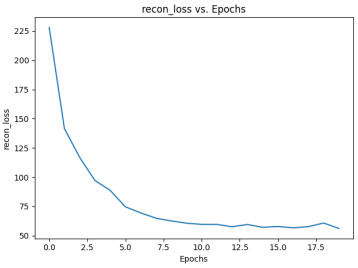
- Question: Follow the instructions in the README.md file in the vae/ folder to complete the implementation of VAEs.
- Debugging Tips:
  - Make sure the auto-encoder can produce good-quality reconstructions before moving on to the VAE. While the VAE reconstructions might not be clear and the VAE samples even less so, the auto-encoder reconstructions should be very clear.
  - If you are struggling to get the VAE portion working: debug the KL loss independently of the reconstruction loss to ensure the learned distribution matches standard normal.
- Expected results:
  - AE: reconstruction loss should be < 40, reconstructions should look similar to original image.
  - VAE: reconstruction loss should be < 145 ($\beta = 1$ case).
  - VAE: reconstruction loss should be < 125 when annealing β.
- Deliverables: The code will log plots to different folders in vae. Please paste the plots into the appropriate place for the questions below. Note for ALL questions, use the reconstructions and samples from the final epoch (epoch 19).

1. Autoencoder: For each latent size, paste your plot of the reconstruction loss curve and reconstructions.

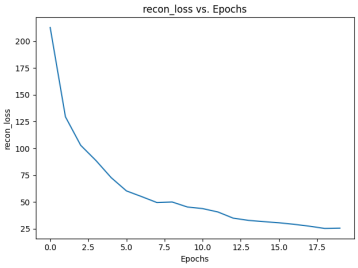
Solution:



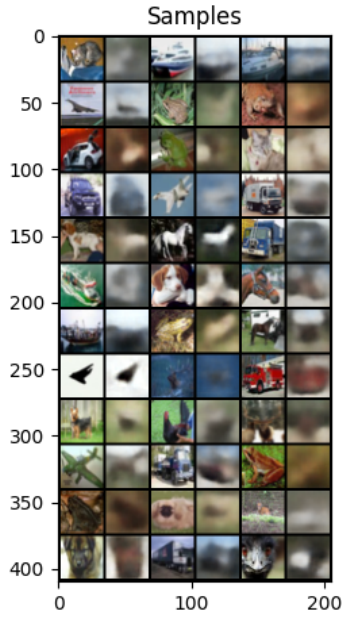
(a) Loss (size 16)



(b) Loss: (size 128)



(c) Loss: (size 1024)



(d) Reconstructions: (size 16)



(e) Reconstructions: (size 128)

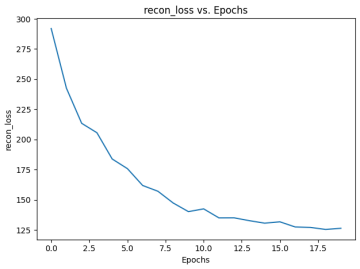


(f) Reconstructions: (size 1024)

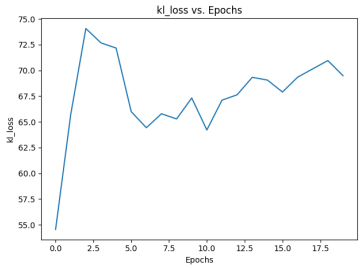
The loss curves show lower reconstruction loss as the latent size increases. At epoch 19, evaluation loss was 203.39 for size 16, 56.13 for size 128, and 25.56 for size 1024. Each input image is followed by its reconstruction. Size 16 preserves broad colors but loses most details. Size 128 retains more object structure, while size 1024 gives the closest reconstructions. The larger latent space can retain more image information, so I use size 1024 for the VAE experiments.

2. VAE: Choose the β that results in the best sample quality, β^*. Report the β^* you used in your experiments. In addition, paste the reconstruction, KL loss curve plots, and the sample images corresponding to the VAE trained using constant β^* and the VAE trained using β annealing scheme with β^*.

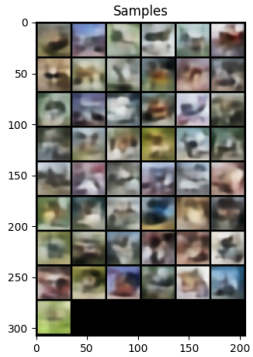
Solution:
β^* value: 0.8.



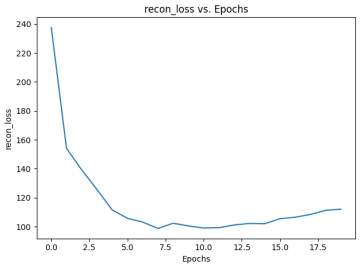
(a) Recon. Loss: constant β^*



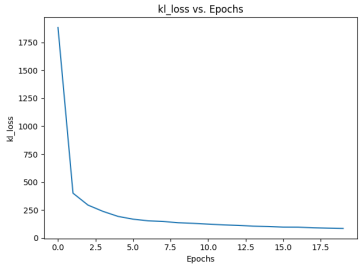
(b) KL Loss: constant β^*



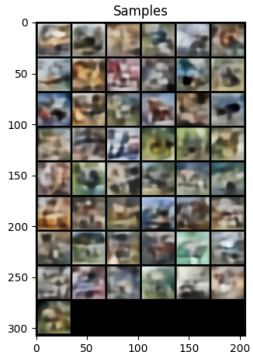
(c) Samples: constant β^*



(d) Recon. Loss: β annealed



(e) KL Loss: β annealed



(f) Samples: β annealed

I tested $\beta = 0.8$, 1.0, and 1.2 using latent size 1024. I selected 0.8 because its final sample grid retained slightly clearer object boundaries, although all three grids remained blurry.

With constant $\beta = 0.8$, evaluation reconstruction loss decreased to 126.46, while KL loss fluctuated and finished at 69.49. With annealing, reconstruction loss fell faster early on, then rose as the KL penalty increased. It finished at 112.03, with KL loss 84.27. The initially weak penalty lets the model focus on reconstruction; increasing it then reduces the large early KL loss. The annealed samples show somewhat more structured shapes, but the improvement is modest and fine details are still missing. The supplied schedule targets 0.8 at epoch 20, so its value at the final plotted epoch, 19, is 0.76.

3 Diffusion Models (20 points)

NOTE: You only need to complete ONE of the following two sections: either Section 3 (Diffusion Models) OR Section 4 (Flow Matching Models). Both sections are worth 20 points.

We will be running inference using a pre-trained diffusion model (DDPM) on CIFAR-10.

- Setup: Download our pre-trained checkpoint for DDPM from here.
- Question: Follow the instructions in the README.md file in the diffusion/ folder to complete the implementation of the sampling procedures for Diffusion Models.
- Expected results:
  - FID of less than 60 for DDPM and DDIM
- Deliverables: The code will log plots to diffusion/data_ddpm and diffusion/data_ddim. Extract plots and paste them into the appropriate section below.

1. Paste your plots of the DDPM and DDIM samples.

   Solution: I completed the flow-matching option in Section 4.

2. Paste in the FID score you obtained from running inference using DDPM and DDIM.

   Solution: I completed the flow-matching option in Section 4.

3. Compare the generated samples from your VAE and diffusion models. What do you observe? Does one generate better pictures than the other? Does the quality of the generated images match with the FID differences?

   Solution: I completed the flow-matching option in Section 4.

4 Flow Matching Models (20 points)

NOTE: You only need to complete ONE of the following two sections: either Section 3 (Diffusion Models) OR Section 4 (Flow Matching Models). Both sections are worth 20 points.

We will be running inference using a pre-trained flow matching model (flow matching) on CIFAR-10.

- Setup: Download our pre-trained checkpoint for flow matching model from here.
- Question: Follow the instructions in the README.md file in the flow_matching/ folder to complete the implementation of the sampling procedures for Flow Matching Models.
- Expected results:
  - FID of less than 60 for Euler ODE sampling
- Deliverables: The code will log plots to flow_matching/flow_euler. Extract the plot and paste it into the section below.

1. Paste your plots of the Euler ODE samples.

Solution:

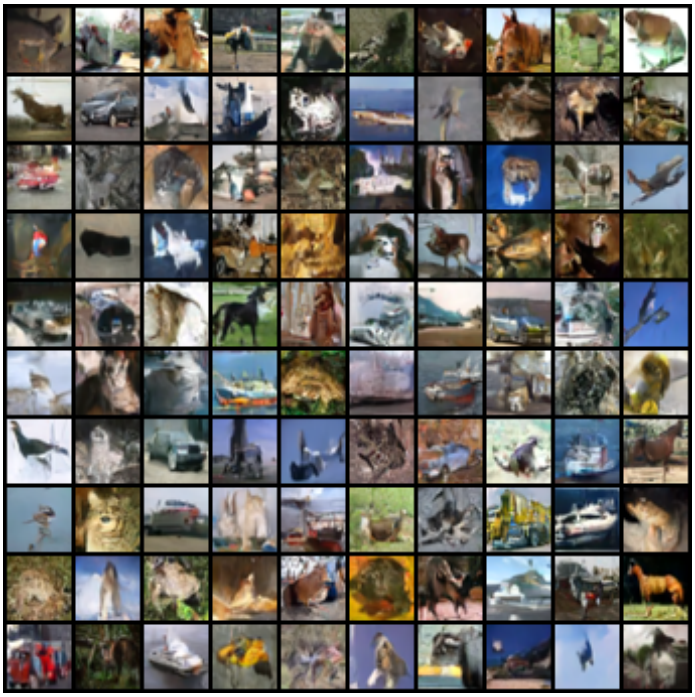


Figure 4.1: Flow Matching Euler ODE Samples

2. Paste in the FID score you obtained from running inference using Euler ODE sampling.

Solution:

- Flow Matching FID: 30.91 (10,000 generated samples).

3. Compare the generated samples from your VAE and flow matching models. What do you observe? Does one generate better pictures than the other?

Solution: The flow matching samples have clearer object boundaries and more texture than the VAE samples. Cars, aeroplanes, and animal bodies are easier to recognize in the flow matching grid. The VAE grids retain broad colors, but many objects blend into their backgrounds and lack fine detail.

Flow matching produces better pictures in this comparison. However, some generated shapes are still distorted, and several small objects remain difficult to identify.

Collaboration Survey Please answer the following:

1. Did you receive any help whatsoever from anyone in solving this assignment?

   ◯ Yes

   ◯ No

   - If you answered 'Yes', give full details:
   - (e.g. "Jane Doe explained to me what is asked in Question 3.4")

2. Did you give any help whatsoever to anyone in solving this assignment?

   ◯ Yes

   ◯ No

   - If you answered 'Yes', give full details:
   - (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

3. Note that copying code or writeup even from a collaborator or anywhere on the internet violates the Academic Integrity Code of Conduct.